

# Informe de Diseño PD

## Ejercicio 1:

### Principios de Diseño SOLID

1. Principio de Responsabilidad Única: Cada clase en el código tiene una única responsabilidad bien definida:

- Buque: Se encarga de representar las características y operaciones específicas de un buque, como reparación y gestión de su estado.
- FlotaBase: Se encarga de administrar una flota de buques, incluyendo operaciones como agregar buques, listarlos, y gestionar reparaciones.
- Observador: Encargado de recibir notificaciones de los cambios en los buques y flotas.
- CrearFlota: Es una implementación concreta de FlotaBase, definiendo el comportamiento específico para una flota creada.

2. Principio de Abierto/Cerrado (OCP - Open/Closed Principle):

- El sistema está diseñado para ser extensible sin necesidad de modificar el código existente. Por ejemplo, se pueden agregar nuevos tipos de flotas extendiendo FlotaBase o nuevas funcionalidades mediante nuevas clases que implementen Sujeto u ObservadorInterfaz sin modificar las clases existentes.

3. Principio de Sustitución de Liskov (LSP - Liskov Substitution Principle):

- Las clases derivadas pueden sustituir a sus clases base sin afectar el correcto funcionamiento del programa. Por ejemplo, CrearFlota hereda de FlotaBase y puede ser utilizada en lugar de FlotaBase sin problemas.

4. Principio de Segregación de Interfaces:

- Las interfaces Flota, Sujeto y ObservadorInterfaz son específicas y limitadas a lo que realmente necesitan:
  - Sujeto: Maneja los métodos de suscripción y notificación.
  - ObservadorInterfaz: Define un único método update para recibir notificaciones.
  - Flota: Define los métodos relacionados con la gestión de buques.

5. Principio de Inversión de Dependencias (DIP - Dependency Inversion Principle):

- Las clases dependen de abstracciones en lugar de implementaciones concretas. Por ejemplo, la clase Buque y FlotaBase dependen de ObservadorInterfaz en lugar de una clase concreta, lo que facilita el desacoplamiento y la extensibilidad.

### Patrón de Diseño Utilizado

Patrón Observador (Observer Pattern)

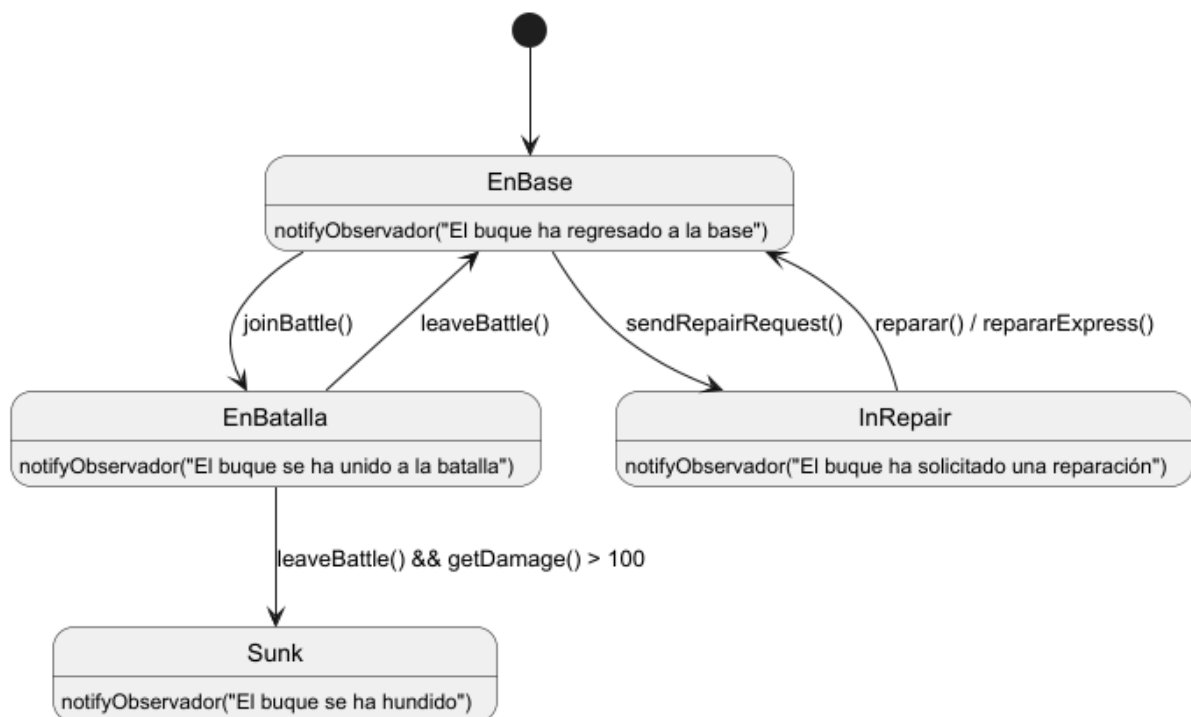
- Explicación del Patrón:
  - El patrón Observer permite que un objeto (sujeto) notifique a otros objetos (observadores) cuando ocurre un cambio en su estado. Este patrón facilita la comunicación entre objetos sin que estos estén directamente acoplados.

- Se utiliza para implementar un sistema de notificaciones en el que los cambios en los buques y flotas son comunicados automáticamente a los observadores interesados.
- Clases Involucradas:
  - Sujeto (Subject): Interfaz Sujeto, implementada por Buque y FlotaBase. Proporciona métodos para agregar, eliminar y notificar observadores.
  - Observador (Observer): Interfaz ObservadorInterfaz, implementada por la clase Observador.
  - Concreto Sujeto (Concrete Subject): Clases Buque y CrearFlota que notifican a los observadores cuando su estado cambia.
  - Concreto Observador (Concrete Observer): La clase Observador que recibe y muestra las notificaciones.

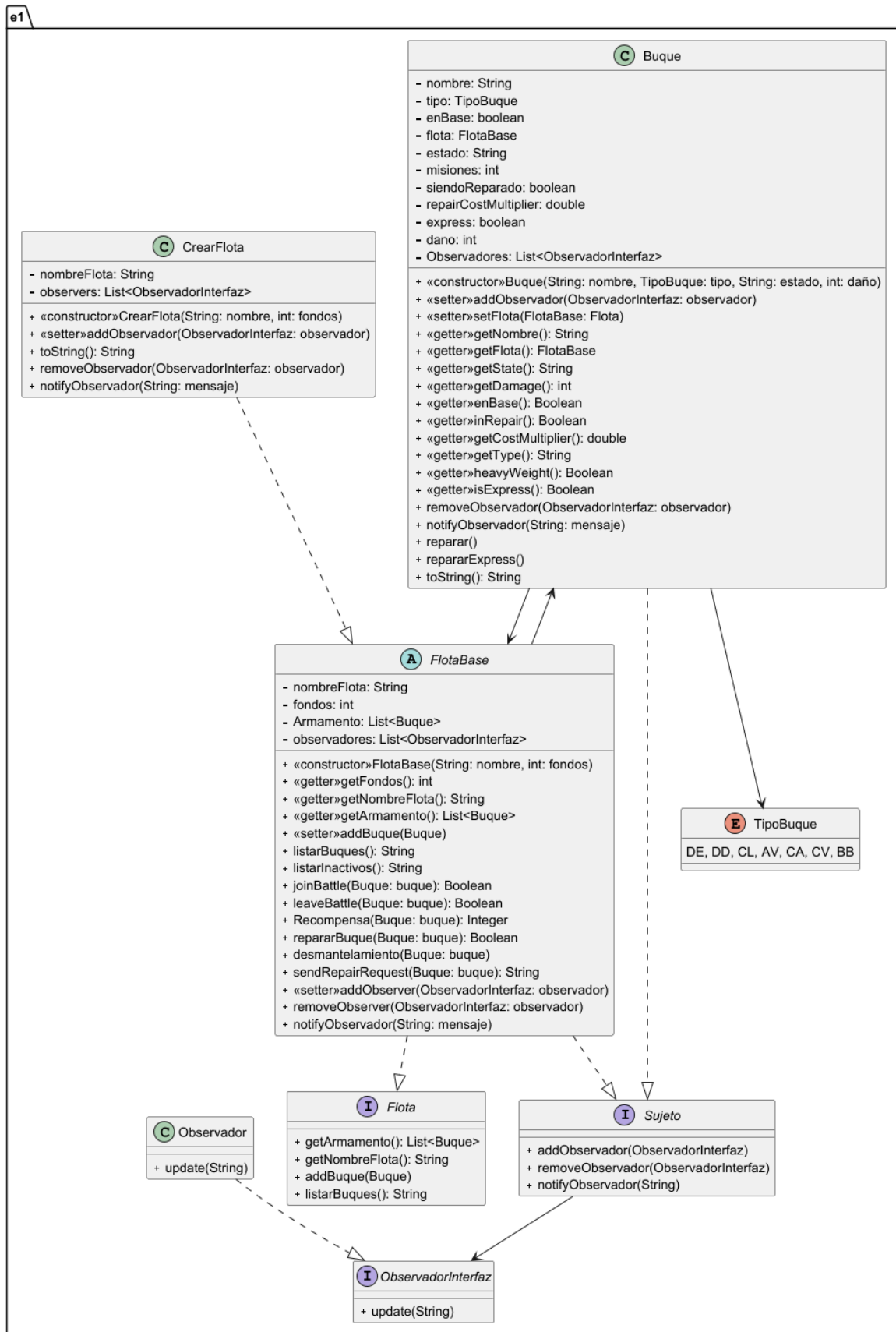
#### 1. Justificación de su uso:

- Se utiliza el patrón Observer para permitir que múltiples observadores puedan suscribirse a eventos relacionados con los buques y flotas. Esto facilita el seguimiento de cambios sin que las clases Buque y FlotaBase necesiten conocer los detalles de los observadores.

## Diagrama UML de estados



# Diagrama UML estático de clases



## Ejercicio 2:

### Principios SOLID usados:

#### 1. Principio de Responsabilidad Única (SRP):

- Cada clase tiene una única responsabilidad:
- "AccionBase" encapsula los datos de una acción.
- "MostrarDatosDetallados" y "MostrarDatosSencillos" manejan diferentes formas de presentar los datos.
- "Cliente" solo gestiona la interacción con las estrategias de presentación.

#### 2. Principio Abierto-Cerrado (OCP):

- Puedes agregar nuevos tipos de estrategias ("MostrarDatos") sin modificar el código existente de "Cliente".

- Ejemplo: Agregar una estrategia "MostrarDatosResumidos" solo requeriría implementar la interfaz "MostrarDatos".

#### 3. Principio de Sustitución de Liskov (LSP):

- Las implementaciones de "MostrarDatos" ("MostrarDatosDetallados" y "MostrarDatosSencillos") pueden sustituir a la interfaz "MostrarDatos" sin modificar el comportamiento de "Cliente".
- Esto asegura que cualquier estrategia es válida y funcional dentro del contexto de "Cliente".

#### 4. Principio de Segregación de Interfaces (ISP):

- La interfaz "MostrarDatos" define un contrato claro y específico. Cada implementación (estrategia) se ocupa solo de la funcionalidad necesaria para mostrar datos.

#### 5. Principio de Inversión de Dependencias (DIP):

- "Cliente" depende de la abstracción ("MostrarDatos") y no de implementaciones concretas ("MostrarDatosDetallados" o "MostrarDatosSencillos"). Esto reduce el acoplamiento y aumenta la flexibilidad.

## Patrón de diseño usado:

### Patrón Estrategia

- **¿Dónde se aplica?**

“MostrarDatos” es una interfaz que define un contrato común para mostrar datos de una acción.

Las implementaciones concretas, como “MostrarDatosDetallados” y “MostrarDatosSencillos”, encapsulan diferentes estrategias para mostrar los datos.

|                                                 |   |                                                |
|-------------------------------------------------|---|------------------------------------------------|
| MostrarDatos                                    |   | Interfaz común para encapsular los algoritmos. |
| MostrarDatosDetallados<br>MostrarDatosSencillos | y | Estrategias concretas.                         |
| Cliente                                         |   | Contexto que utiliza una estrategia concreta   |

- **Beneficios:**

Se separa el comportamiento de “MostrarDatos” de la lógica principal de “Cliente”.

Se pueden agregar fácilmente nuevos tipos de comportamiento (estrategias) sin modificar las clases existentes, cumpliendo el Principio Abierto-Cerrado (OCP).

- **¿Por qué es adecuado?**

Los clientes tienen diferentes necesidades para mostrar los datos y el patrón Estrategia permite manejar esta variación de manera extensible y correcta.

Diagrama UML de clases:

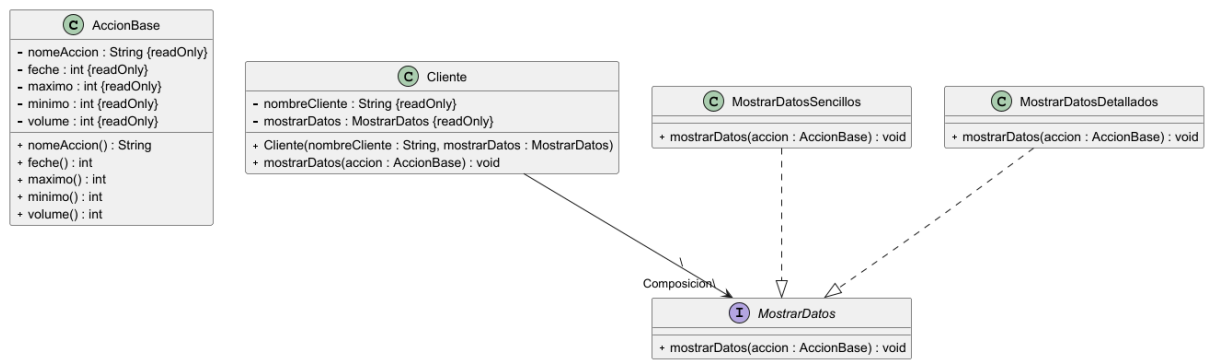


Diagrama UML de secuencia:

